2027-7-24 Linux库--ELF与地址空间

- 1. ELF 是库和可执行程序的共同容器
  - 1.1 ELF 文件的基本概念
    - 1.1.1 为什么库也属于 ELF
    - 1.1.2 ELF 文件的四个主要部分
  - 1.2 从文件偏移理解 ELF
    - 1.2.1 ELF 文件本质上也是一个字节数组
    - 1.2.2 ELF Header 是后续解析的入口
- 2. 使用 readelf -h 读取 ELF Header
  - 2.1 ELF Header 的关键字段
    - 2.1.1 文件身份与运行平台
    - 2.1.2 入口地址和各类表的位置
    - 2.1.3 可重定位文件与可执行文件的差异
- 3. Section 与 Segment 的对应关系
  - 3.1 Section：链接器使用的细粒度视图
    - 3.1.1 用 readelf -S 查看节头表
    - 3.1.2 常见 Section 的作用
  - 3.2 Segment：加载器使用的执行视图
    - 3.2.1 用 readelf -l 查看程序头表
    - 3.2.2 为什么要把多个 Section 合并成 Segment
- 4. 虚拟地址与进程地址空间
  - 4.1 程序未加载时也有地址
    - 4.1.1 平坦模式下的统一编址
    - 4.1.2 虚拟地址并不等于物理地址
  - 4.2 进程地址空间的典型布局
    - 4.2.1 从低地址到高地址
    - 4.2.2 mm_struct、vm_area_struct 与 ELF 的关系
- 5. 从 ELF 文件到进程运行时
  - 5.1 可执行文件的加载过程
    - 5.1.1 加载器读取程序头并建立映射
    - 5.1.2 程序头和接口路径共同决定运行方式
  - 5.2 动态库如何进入进程地址空间
    - 5.2.1 动态库映射到共享区
    - 5.2.2 一个物理副本，多份虚拟映射
- 6. 用命令验证 ELF 结构
  - 6.1 建立一条完整的观察路径

- 6.1.1 逐步查看文件、程序头、节头和反汇编
- 6.1.2 对照文件内容与进程布局

1. ELF 是库和可执行程序的共同容器

1.1 ELF 文件的基本概念

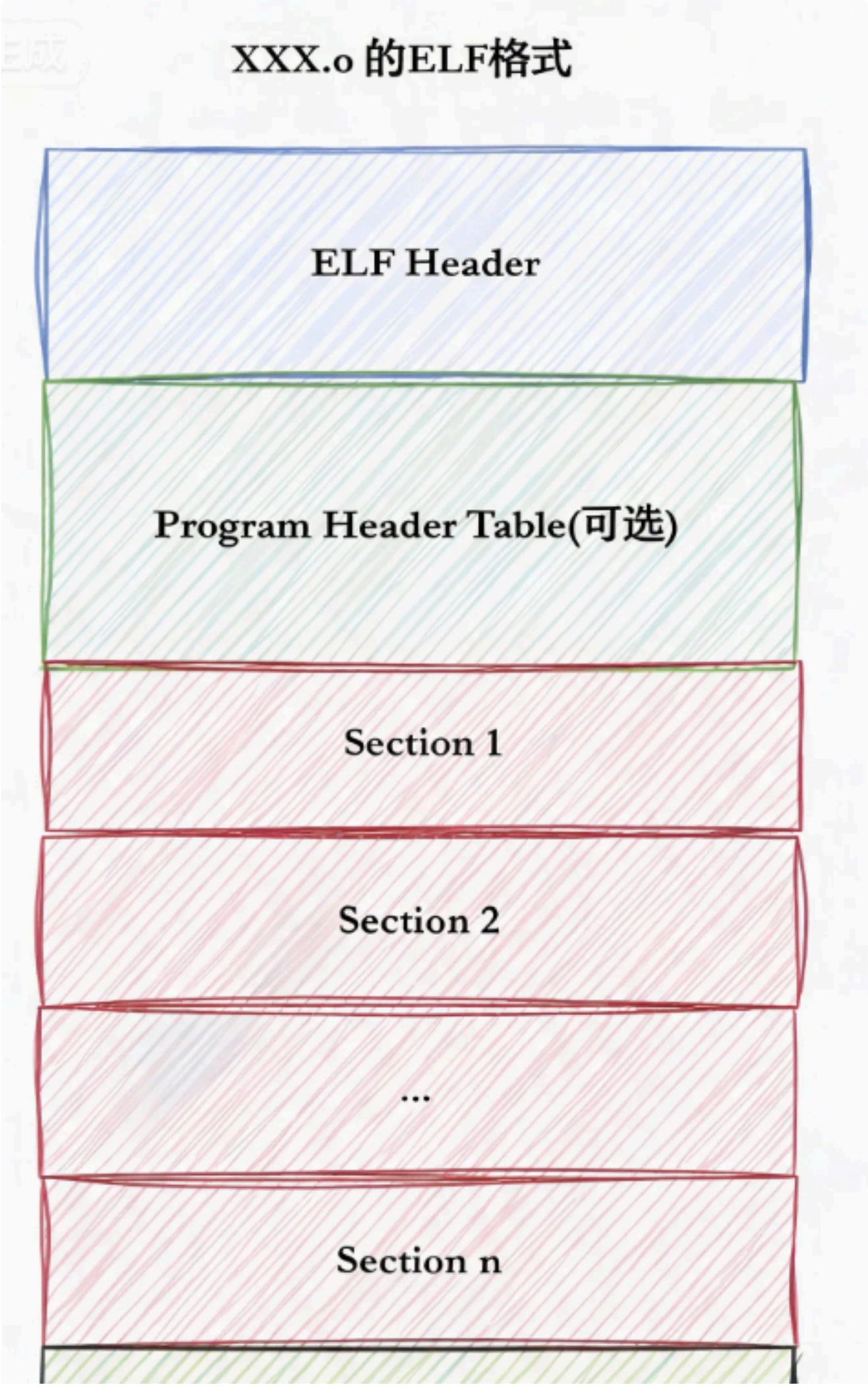
1.1.1 为什么库也属于 ELF

可执行程序不是一个可以把代码和数据随意拼在一起的普通二进制文件，它需要固定结构，使链接器和操作系统加载器都能理解其中的内容。Linux 使用 ELF（Executable and Linkable Format，可执行与可链接格式）封装目标文件、可执行文件和共享库。

常见的 ELF 文件包括：

类型	扩展名	作用
可重定位文件	`.o`	等待与其他目标文件链接
可执行文件	例如 `a.out`	可以创建并启动进程
共享目标文件	`.so`	运行时加载的动态库
内核转储文件	core dump	保存进程异常时的执行现场

`.o`、`.so` 和可执行程序能够进入同一套链接、加载流程，就是因为它们都遵循 ELF 格式。静态库 `.a` 本身是多个 `.o` 的归档，真正参与链接的仍然是其中的 ELF 目标文件。


XXX.o 的ELF格式
ELF Header
Program Header Table(可选)
Section 1
Section 2
...
Section n



1.1.2 ELF 文件的四个主要部分

一个 ELF 文件通常可以看作下列区域：

```
ELF Header
Program Header Table
Section 1
Section 2
...
Section n
Section Header Table
```

区域	主要作用
ELF Header	描述文件类型，并记录各表的起始位置、入口地址等信息
Program Header Table	告诉加载器哪些段需要映射、如何映射以及具有什么权限
Section	存放代码、数据、只读常量、符号表等具体内容
Section Header Table	描述每个 section 的名称、类型、地址、偏移和大小

Program Header Table 在可重定位文件 `.o` 中可以为空，在可执行文件和共享库中通常存在，因为后两者需要由加载器映射到进程地址空间。

1.2 从文件偏移理解 ELF

1.2.1 ELF 文件本质上也是一个字节数组

ELF 文件的各个结构并不是抽象存在，而是位于文件中的确定偏移处。ELF Header 就像文件目录，记录其他区域的起始位置和长度：

```
文件起始位置
    |
    +-- ELF Header
    |
    +-- e_phoff 指向 Program Header Table
    |
    +-- 各个 Section 的字节内容
    |
    +-- e_shoff 指向 Section Header Table
```

Section Header 中同样保存 `Offset`、`Address`、`Size` 等信息。因此，查找某个 section 时，可以先从节头表中取得偏移，再把它当作文件字节数组中的下标，找到具体内容。

1.2.2 ELF Header 是后续解析的入口

ELF Header 位于文件开头，主要任务是识别文件并定位其他区域。后续使用 `readelf` 查看程序头、节头、符号表时，都依赖 ELF Header 中记录的位置和条目数量。

2. 使用 `readelf -h` 读取 ELF Header

2.1 ELF Header 的关键字段

2.1.1 文件身份与运行平台

```
readelf -h a.out
```

前几个字段用于确认文件确实是当前系统能够识别的 ELF：

字段	含义
`Magic`	ELF 魔数，通常以 `7f 45 4c 46` 开头
`Class`	ELF32 或 ELF64
`Data`	字节序，常见为小端序
`Version`	ELF 版本
`OS/ABI`	操作系统应用二进制接口
`Type`	可重定位、可执行、共享对象或 core 文件
`Machine`	目标 CPU 架构，例如 x86-64

`Type` 常见取值如下：

取值	含义
`REL`	可重定位文件，例如 `.o`
`EXEC`	传统非 PIE 可执行文件
`DYN`	共享对象，或者现代编译器生成的 PIE 可执行文件
`CORE`	内存转储文件

现代 `gcc` 在部分系统中默认生成 PIE 可执行文件，因此可执行程序的 `Type` 也可能显示为 `DYN (Position-Independent Executable file)`，不能只凭 `DYN` 就认定它一定是 `.so`。

```
ZL@VM-0-14-ubuntu:~/code/lesson20-lib$ readelf -h a.out
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  Class:                             ELF64
  Data:                              2's complement, little endian
  Version:                           1 (current)
  OS/ABI:                            UNIX - System V
  ABI Version:                       0
  Type:                              DYN (Position-Independent Executable file)
  Machine:                           Advanced Micro Devices X86-64
  Version:                           0x1
  Entry point address:               0x1060
  Start of program headers:          64 (bytes into file)
  Start of section headers:          13976 (bytes into file)
  Flags:                             0x0
  Size of this header:               64 (bytes)
  Size of program headers:           56 (bytes)
  Number of program headers:         13
  Size of section headers:           64 (bytes)
  Number of section headers:         31
  Section header string table index: 30
```

2.1.2 入口地址和各类表的位置

ELF Header 中与加载关系最密切的字段包括：

字段	含义
`Entry point address`	CPU 开始执行程序时使用的入口虚拟地址
`Start of program headers`	Program Header Table 在文件中的偏移
`Start of section headers`	Section Header Table 在文件中的偏移
`Size of program headers`	一个程序头条目的大小
`Number of program headers`	程序头条目的数量

字段	含义
`Size of section headers`	一个节头条目的大小
`Number of section headers`	节头条目的数量

入口地址并不是 `main` 函数地址。程序真正执行的第一段代码通常来自 C 运行库或链接器提供的 `_start`，经过运行时初始化后，最终才会调用 `main`。

2.1.3 可重定位文件与可执行文件的差异

文件	入口地址	Program Header Table	主要用途
`.o`	通常为 `0x0`	通常不存在或数量为 0	等待链接
可执行文件	有有效入口地址	存在	创建进程并加载
`.so`	可以有入口	存在	映射到进程共享区

`.o` 还不具备完整的加载布局，链接器需要先合并其中的 section、处理符号和地址重定位，才能形成可以被操作系统加载的可执行文件。

3. Section 与 Segment 的对应关系

3.1 Section：链接器使用的细粒度视图

3.1.1 用 `readelf -S` 查看节头表

```
readelf -S a.out
```

节头表把文件拆分成功能更加明确的 section。每个条目记录名称、类型、虚拟地址、文件偏移、大小、权限和归属关系。

```
ZL@VM-0-14-ubuntu:~/code/lesson20-lib$ readelf -S a.out
There are 31 section headers, starting at offset 0x3698:

Section Headers:
  [Nr] Name              Type             Address           Offset
       Size              EntSize          Flags  Link  Info  Align
  [ 0]                   NULL             0000000000000000  00000000
       0000000000000000  0000000000000000           0     0     0
  [ 1] .interp           PROGBITS         0000000000000318  00000318
       000000000000001c  0000000000000000   A       0     0     1
  [ 2] .note.gnu.pr[...] NOTE             0000000000000338  00000338
       0000000000000030  0000000000000000   A       0     0     8
  [ 3] .note.gnu.bu[...] NOTE             0000000000000368  00000368
       0000000000000024  0000000000000000   A       0     0     4
  [ 4] .note.ABI-tag     NOTE             000000000000038c  0000038c
       0000000000000020  0000000000000000   A       0     0     4
  [ 5] .gnu.hash         GNU_HASH         00000000000003b0  000003b0
       0000000000000024  0000000000000000   A       6     0     8
  [ 6] .dynsym           DYNSYM           00000000000003d8  000003d8
       00000000000000a8  0000000000000018   A       7     1     8
  [ 7] .dynstr           STRTAB           0000000000000480  00000480
       000000000000008f  0000000000000000   A       0     0     1
  [ 8] .gnu.version      VERSYM           0000000000000510  00000510
       000000000000000e  0000000000000002   A       6     0     2
  [ 9] .gnu.version_r    VERNEED          0000000000000520  00000520
       0000000000000030  0000000000000000   A       7     1     8
  [10] .rela.dyn         RELA             0000000000000550  00000550
       00000000000000c0  0000000000000018   A       6     0     8
  [11] .rela.plt         RELA             0000000000000610  00000610
       0000000000000018  0000000000000018  AI       6    24     8
  [12] .init             PROGBITS         0000000000001000  00001000
       000000000000001b  0000000000000000  AX       0     0     4
```

3.1.2 常见 Section 的作用

Section	主要内容	常见权限
`.text`	机器指令	可读、可执行
`.rodata`	字符串等只读数据	只读
`.data`	已初始化的全局变量和静态变量	可读、可写
`.bss`	未初始化数据预留的内存	可读、可写
`.symtab`	符号名与地址的对应关系	文件和链接使用
`.dynsym`	动态链接使用的符号表	动态链接使用
`.got` / `.got.plt`	动态符号跳转地址	加载或运行时修改

`.bss` 通常采用 `NOBITS`，在文件中不保存大量零字节；加载器根据内存大小创建相应区间并清零。

3.2 Segment：加载器使用的执行视图

3.2.1 用 `readelf -l` 查看程序头表

```
readelf -l a.out
```

程序头表描述的是 segment，而不是细粒度的 section。常见程序头类型包括：

类型	作用
`LOAD`	需要从文件映射到进程地址空间的段
`INTERP`	指定动态链接器的路径
`DYNAMIC`	动态链接、重定位和依赖信息
`GNU_STACK`	指定栈是否可执行
`GNU_RELRO`	加载后将部分区域改为只读

每个 `LOAD` 段实际表示为：

```
文件偏移 Offset
虚拟地址 VirtAddr
文件大小 FileSiz
内存大小 MemSiz
访问权限 Flags
对齐 Align
```

加载器按照这些字段，把文件中从 `Offset` 开始的 `FileSiz` 字节映射到从 `VirtAddr` 开始的地址；如果 `MemSiz > FileSiz`，多出来的内存区域会被清零。这正好对应 `.bss` 只预留空间、文件不保存全部零字节的特点。

3.2.2 为什么要把多个 Section 合并成 Segment

不同 section 的大小和权限可能不同，但有些 section 具有相同访问权限，例如多个可执行 section 都可以合并为一个可读可执行段，多个可写 section 可以合并为一个可读可写段。

页是内存加载和管理的基本单位，常见页大小为 4 KiB。若每个小 section 都单独映射，容易造成页面对齐后的碎片和权限管理开销；按权限合并为 segment 后，可以减少页面浪费，也方便加载器统一设置访问权限。

这也是 ELF 提供两套视图的原因：

视图	依据	主要使用者	关注点
链接视图	Section Header Table	链接器、调试器	功能模块、符号和重定位
执行视图	Program Header Table	操作系统加载器	映射范围、地址、大小和权限

4. 虚拟地址与进程地址空间

4.1 程序未加载时也有地址

4.1.1 平坦模式下的统一编址

编译器会为 ELF 中的代码和数据统一安排地址。查看反汇编时，每行指令左侧就是 ELF 逻辑地址：

```
objdump -S ./a.out
```

在平坦模式下，段基址通常从 0 开始，因此逻辑地址和虚拟地址在很多场景下可以统一理解。一个地址可以写成：

```
逻辑地址 = 段基址 + 偏移量
```

程序尚未加载到物理内存时没有物理地址，但它的代码和数据的虚拟地址关系已经由 ELF 确定。ELF Header 中的入口地址告诉 CPU 应该从哪里开始执行。

4.1.2 虚拟地址并不等于物理地址

进程看到的是自己的虚拟地址空间，CPU 通过页表把虚拟地址转换为物理地址。不同进程可以使用相同的虚拟地址，而它们实际对应的物理页面可以不同。

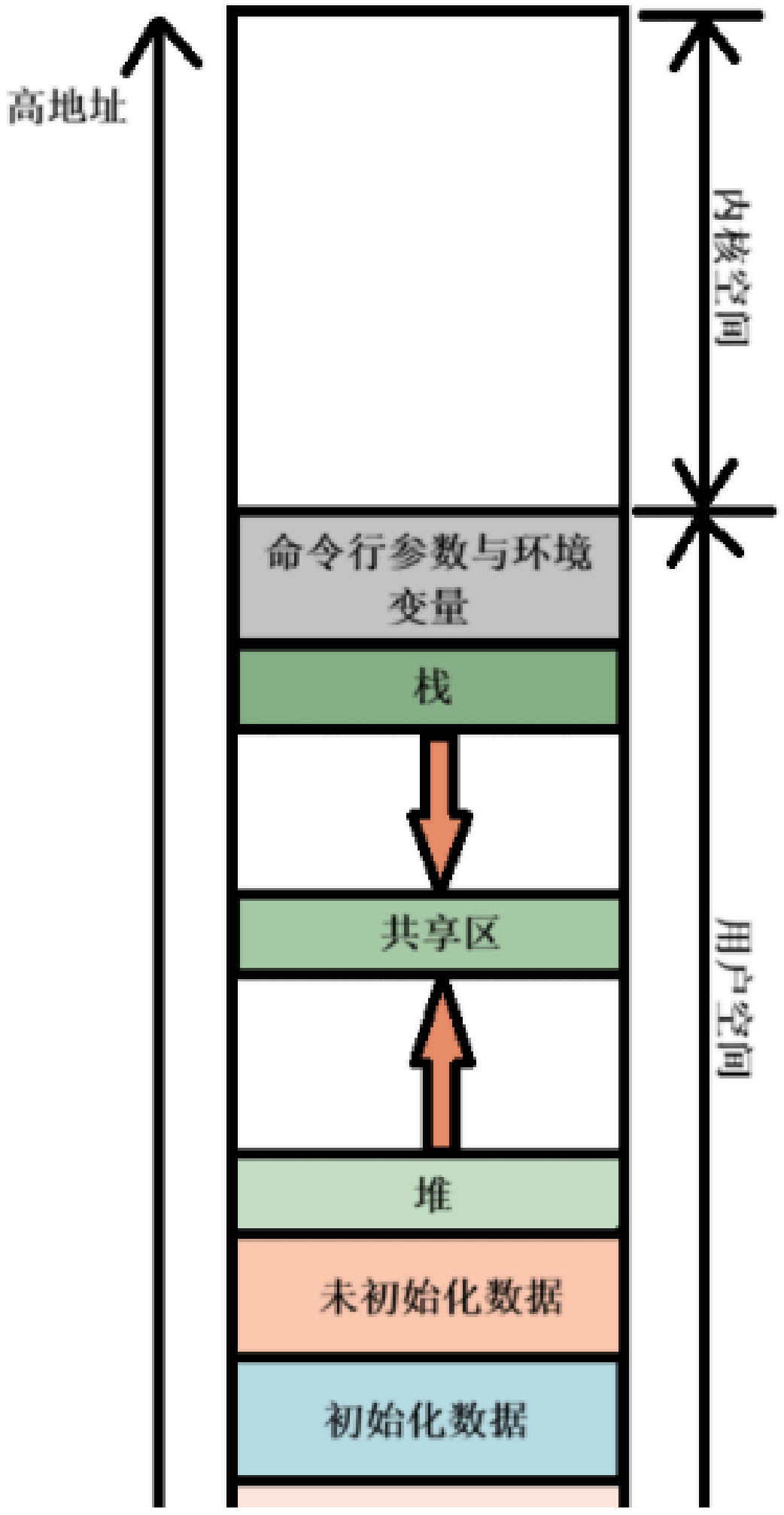
虚拟地址机制既需要操作系统建立进程地址空间和页表，也需要编译器按照约定对 ELF 进行统一编址。

4.2 进程地址空间的典型布局

4.2.1 从低地址到高地址

常见的 32 位 Linux 地址空间示意图如下：

```
高地址
├── 内核空间
├── 命令行参数与环境变量
├── 栈
├── 共享区
├── 堆
├── 未初始化数据
├── 初始化数据
└── 正文代码
低地址
```


高地址
内核空间
用户空间
命令行参数与环境变量
栈
共享区
堆
未初始化数据
初始化数据



> 图中“用户空间 3 GiB、内核空间 1 GiB”是典型的 32 位 Linux 布局。x86-64 的地址范围和划分方式不同，但“代码和数据在低地址、共享区位于中间、栈从高地址向低地址增长”的结构思想仍然适用。

4.2.2 `mm_struct`、`vm_area_struct` 与 ELF 的关系

创建一个进程时，操作系统先建立与进程地址空间相关的数据结构，再加载 ELF 文件。每个进程都有自己的 `mm_struct`，其中维护页表等信息；`vm_area_struct` 用 `[start, end]` 等范围描述代码区、数据区、堆、栈、共享区等虚拟内存区域。

这些区域的起始地址、范围和权限并非凭空产生，而是来自 ELF 的各个 segment。加载器根据程序头建立虚拟内存区域，再通过页表把虚拟地址映射到实际物理页面。

5. 从 ELF 文件到进程运行时

5.1 可执行文件的加载过程

5.1.1 加载器读取程序头并建立映射

加载可执行程序时可以按以下主线理解：

1. 创建进程相关的内核数据结构，例如 `mm_struct` 和虚拟内存区域。
2. 打开 ELF 文件，读取 ELF Header，找到 Program Header Table。
3. 遍历 `LOAD` 段，为每个段建立虚拟内存映射。
4. 把段对应的文件内容读入或映射到内存，并为 `.bss` 等区域补零。
5. 将程序起始位置设置为 ELF Header 中的入口地址。
6. 如果存在 `INTERP` 段，先启动指定路径的动态链接器。
7. 完成必要的动态链接和运行时初始化后，从入口开始执行，最终调用 `main`。

5.1.2 程序头和接口路径共同决定运行方式

`readelf -l a.out` 中的 `INTERP` 段可能显示：

```
[Requesting program interpreter: /lib64/ld-linux-x86-64.so.2]
```

这说明程序需要动态链接器参与。程序入口处的代码并不是直接执行 `main`，而是先进入运行时启动流程；动态链接器负责加载共享库、处理依赖并完成必要的重定位。

5.2 动态库如何进入进程地址空间

5.2.1 动态库映射到共享区

动态库 `.so` 也是一个 ELF 文件，需要先被打开，再把其中的代码和数据映射到进程地址空间。它与普通可执行程序的重要区别是：动态库通常被映射到进程的共享区，并由多个进程共用。

5.2.2 一个物理副本，多份虚拟映射

当多个进程使用同一个动态库时：

1. 动态库文件从磁盘加载到物理内存。
2. 每个进程通过自己的 `mm_struct` 和页表，把该动态库映射到自己的共享区。
3. 不同进程的虚拟地址可以不同，但只读代码页可以指向同一份物理页面。
4. 每个进程仍然拥有独立的页表、可写数据和运行时状态。
5. 动态链接器根据每个进程的实际映射情况完成符号解析和地址重定位。

因此，动态库既被称为共享库，又是“进程相关”的：库的机器码可以在物理内存中共享，但它在每个进程中对应的虚拟地址不一定相同。

6. 用命令验证 ELF 结构

6.1 建立一条完整的观察路径

6.1.1 逐步查看文件、程序头、节头和反汇编

```
file ./a.out
readelf -h ./a.out
readelf -l ./a.out
readelf -S ./a.out
```

```
readelf -s ./a.out
objdump -S ./a.out
```

各命令的关注点如下：

命令	主要观察内容
`file`	文件类型、ELF 位数、架构、是否动态链接
`readelf -h`	ELF 类型、机器、入口地址和表的偏移
`readelf -l`	加载段、虚拟地址范围、权限和动态链接器
`readelf -S`	section 的地址、偏移、大小和标志
`readelf -s`	符号名称、类型、定义位置以及是否未定义
`objdump -S`	按虚拟地址展示反汇编和对应源代码

6.1.2 对照文件内容与进程布局

观察时可以把两层信息连接起来：

```
ELF Header
    -> 找到程序头和节头
Program Header
    -> 决定加载器创建哪些 segment 和权限
Section Header
    -> 描述文件内部的代码、数据、符号和重定位结构
mm_struct / vm_area_struct / 页表
    -> 形成进程运行时看到的虚拟地址空间
动态库 .so
    -> 映射到多个进程的共享区，共享物理内存中的库代码
```

这样，ELF 文件中的静态描述、加载到内存后的虚拟地址范围以及动态库的共享关系就能形成一条完整的理解路径。